Utviklingsstatistikk

Oppsummering

Formål

Utfordring med å måle sikker og stabil drift

Måleparametere

Om systemet

Målinger

- Kodestørrelse
- Kodeendringer
- Netto vekst i kodelinjer
- Detaljer i netto vekst i kodelinjer
- Oppdatering av avhengigheter
- Løpende endringer i systemet
- Risiko

Oppsummering

[TBW]

Formål

Teamet har blitt bedt om å måle på noe som kan indikere om vi har sikker og stabil drift.

Utfordring med å måle sikker og stabil drift

Utfordringen med tradisjonell måling av sikker og stabil drift er at den gjerne måler på ting som oppetid eller rettetid. Dette er målinger som først slår ut etter at driften ikke lenger er sikker og/eller stabil, og det kan ta lang tid fra driften faktisk har blitt problematisk til målingene slår ut.

Vår erfaring er at risiko øker når systemer ikke holdes vedlike. Risiko påvirkes av manglende vedlikehold gjennom flere prosesser:

1. Manglende oppdatering av avhengigheter gir økt sikkerhetsrisiko

2. Manglende teknisk oppdatering gir økt sikkerhetsrisiko og risiko for lav kjennskap til utdaterte tekniske løsninger
3. Lite andre endringer reduserer kunnskapen om systemet som øker rettetid og øker sjansen for feil når man en sjelden gang gjør enderinger

Vi mener det er verdt å forsøke å måle på utviklingsprosessen for å se om det kan gi indikatorer som sier noe om risikobildet.

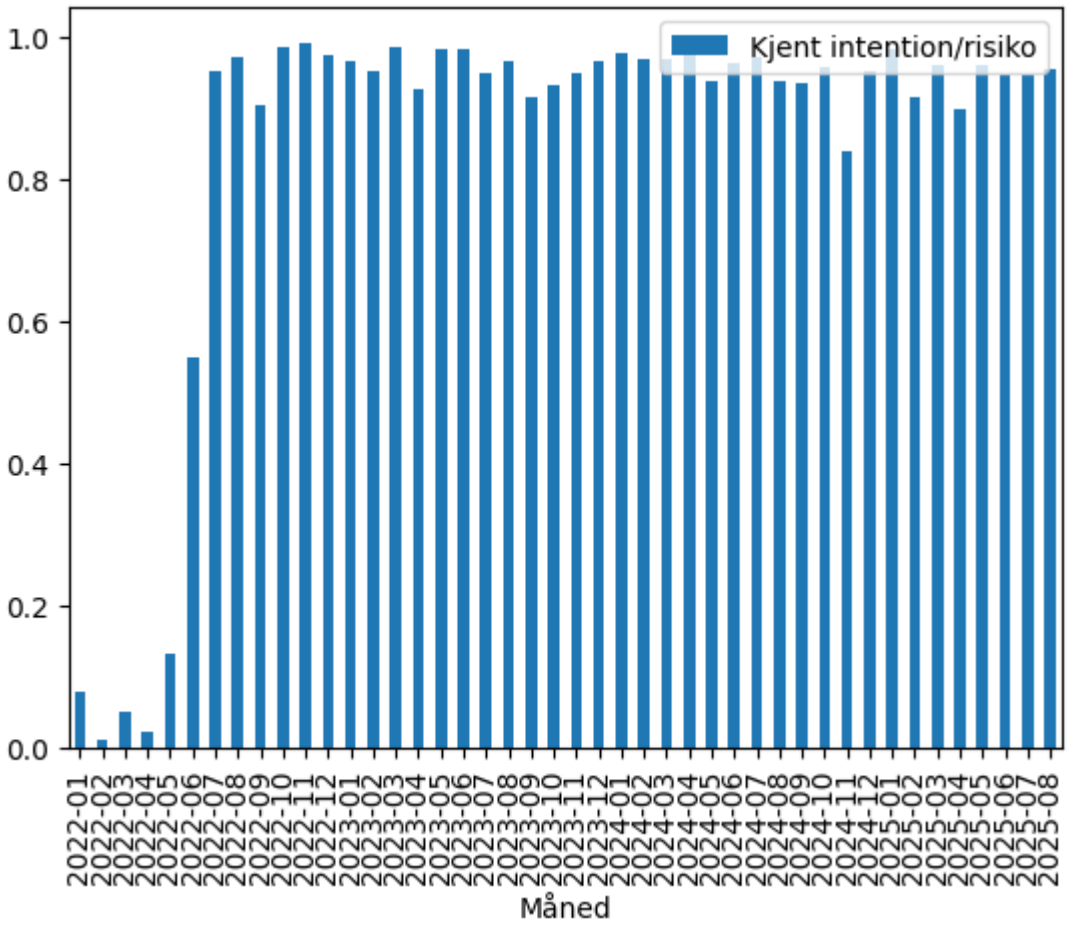
Siden det er kostnader ved å gjøre målinger har vi sett etter målinger som er forholdsvis lette å innhente og som kan gi en indikasjon på at systemet vedlikeholdes.

Måleparametere

Teamet tok sommeren 2022 i bruk teknikkene micro-commits og Arlo's commit notation – dette gjør det mulig å benytte commit-statistikk til å si noe om omfang, intensjon og risiko ved commit'er.

Teamet har besluttet å se om • tellinger og forholdstall på ulike typer commits og • måling på kodestørrelse kan indikere hvordan utviklingen er – og predikere eller illustrere en situasjon der det vil oppstå økt risiko for ustabil / usikker drift.

```
Current date: 2025-08-29 (month 4 of tertial)
Analyzing data from 2022T3 to 2025T2
Comparing 2025T1 vs 2025T2
```



Vi observerer at over 90% av commits er merket med intensjon/risiko - slik har det vært siden juli 2022 - noe som gir et brukbart grunnlag for analyse av commit-aktivitet fra og med 3. tertial i 2022.

Om systemet

Systemet består av en frontendapplikasjon (saksbehandling-ui), 10 backendapplikasjoner, 7 delte bibliotek på backend og et antall bibliotek på frontend (som foreløpig ikke er med i oversikten og statistikken).

Applikasjoner	
saksbehandling-ui	Frontend for saksbehandlere
oppgave	Oppgaver til Oppgave (asynkron)
krav-initialisering	Krav til Pesys (asynkron)
statistikk	Statistikk til Statistikk-folket (asynkron)
onprem-proxy	For å kalle fra GCP til/fra onprem (Pesys mm) (online)
saksbehandling-api	Fasade for diverse API for saksbehandling-ui (online)
begrens-innsyn	Merking av saker med beskyttelse (asynkron)

Applikasjoner	
journalforing	Journalføring av dokumenter (asynkron)
prefill	Preutfylling av SED (online)
fagmodul	Diverse API for saksbehandling-ui m fl (online)
pdl-produsent	Oppdatering av PDL med id'er og adresser (asynkron)

Bibliotek	
ep-metrics	Metrikk-bibliotek
ep-eux	Domene-modell SED/BUC mm
ep-logging	Loggingsbibliotek
ep-personoppslag	Klient for oppslag mot PDL
ep-security-sts	Brukes onprem for token-utveksling
ep-pensjonsinformasjon	Klient for oppslag mot Pesys
ep-kodeverk	Klient mm - opplag i kodeverk
ep-routing	Bibliotek for ruting av oppgave til enhet

Meta	
meta	Verktøy for å jobbe på tvers av modulene nevnt ovenfor
ep-meta-analyse	Verktøy som bygger denne rapporten

Målinger

Kodestørrelse

Kodestørrelse i applikasjoner korrelerer godt med kompleksitet, som igjen korrelerer med feil- og vedlikeholdsomfang.

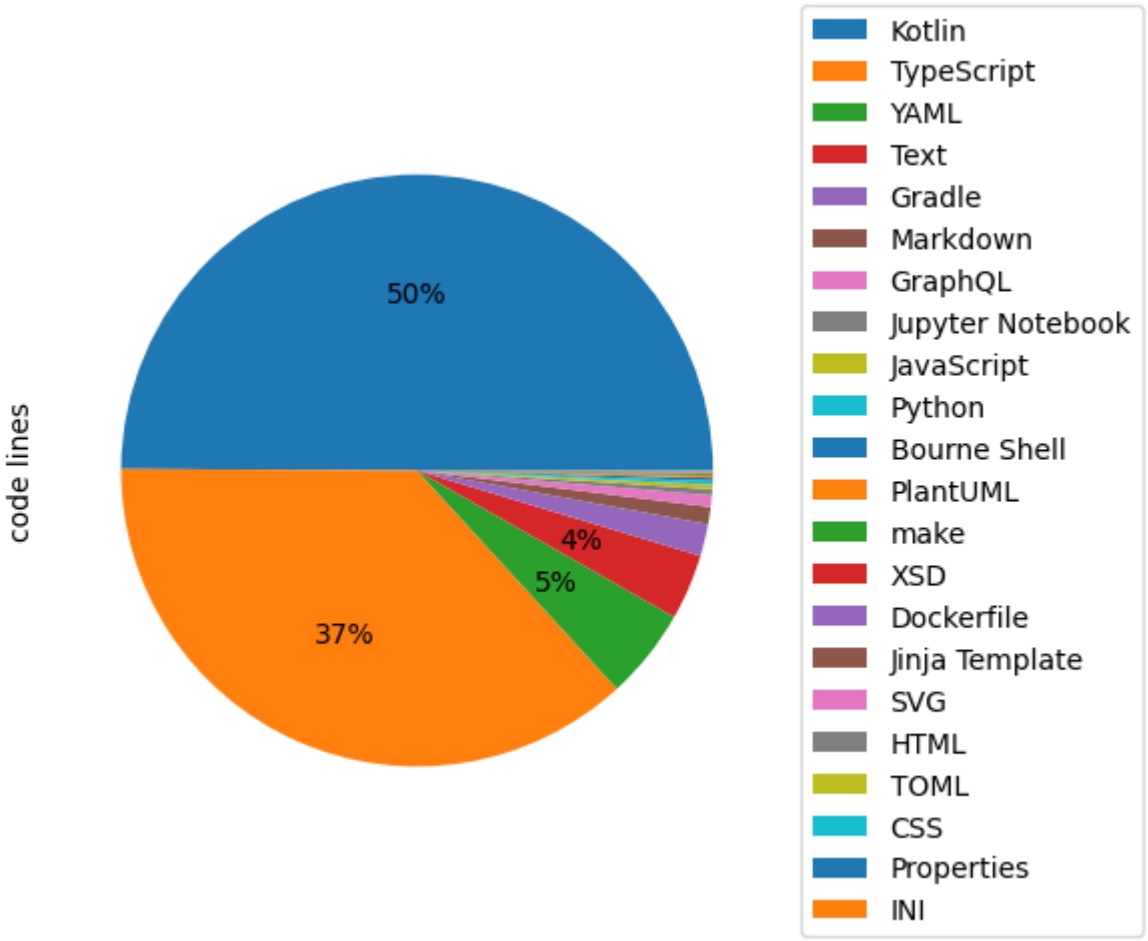
Vi teller kodelinjer med cloc[^1]. (Det er noen små frontend-bibliotek som deles mellom EESSI NAV og EESSI Pensjon som ikke er med i statistikken).

Kodestørrelse

```
Kodestørrelse pr 29.08.2025.
```

Out[9]:

language	files	code lines	comment lines
Kotlin	849	74965	2891
TypeScript	354	55402	1255
YAML	170	7454	498
Text	14	5379	0
Gradle	61	2628	161
Markdown	36	1403	2
GraphQL	9	960	71
Jupyter Notebook	1	486	619
JavaScript	5	378	43
Python	6	295	6
Bourne Shell	20	251	174
PlantUML	3	188	0
make	3	136	0
XSD	3	50	0
Dockerfile	11	44	0
Jinja Template	3	40	0
SVG	4	37	0
HTML	1	27	0
TOML	1	23	0
CSS	1	18	1
Properties	15	15	7
INI	1	11	0



Koden er stort sett skrevet i Kotlin og TypeScript, og noe LESS – øvrig kode er knyttet til bygg, deploy, konfigurasjon og diverse utviklings- og støtteverktøy.

[^1]: Mer om cloc-kommandoen:

```
cloc --csv --vcs git --exclude-dir=dist,build,gradle --
exclude-list-file=.clocignore --exclude-lang=JSON,XML --quiet
.
```

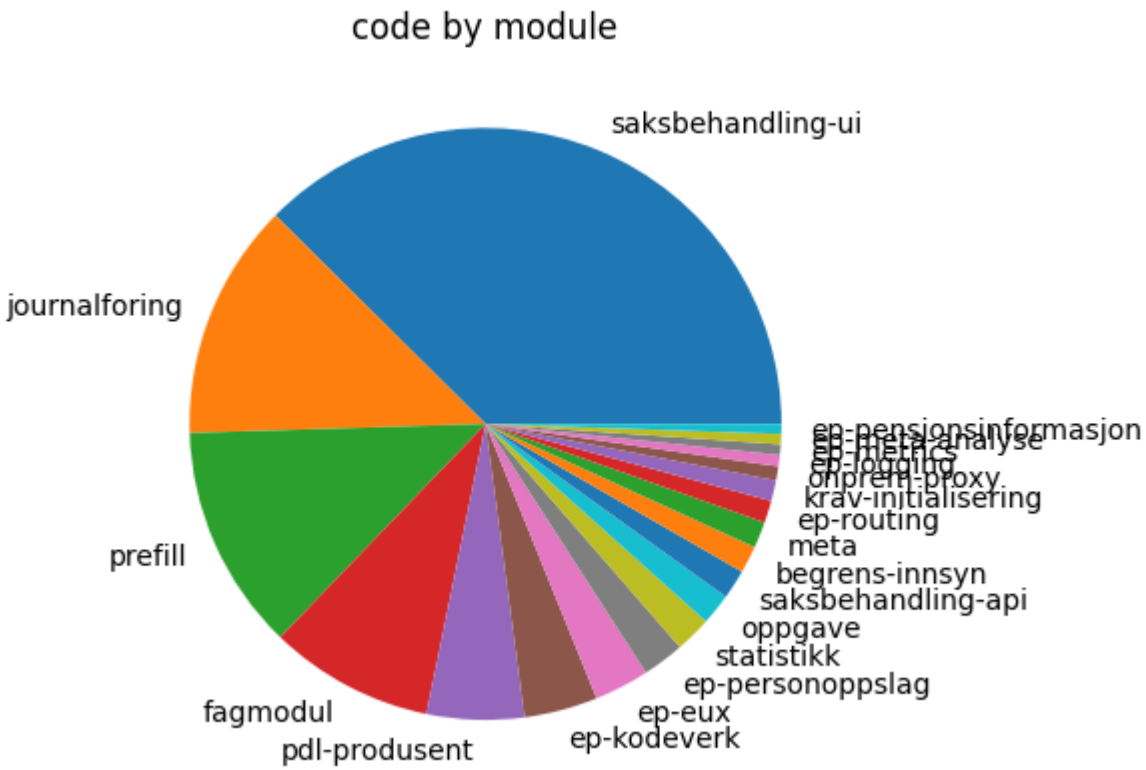
der .clocignore inneholder:

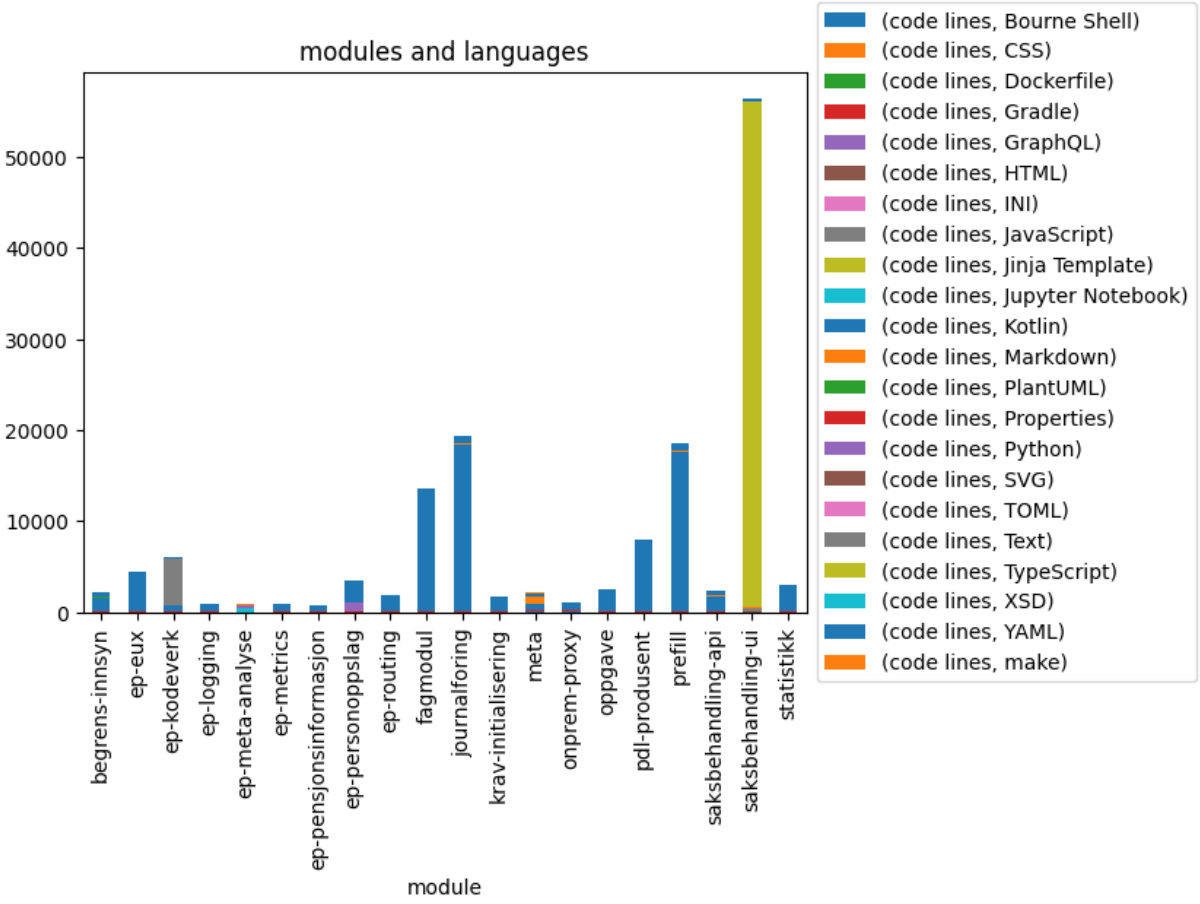
```
gradlew
gradlew.bat
public/pdf.worker.js
public/static/js/pdf.worker.js
package-lock.json
src/components/PostalCodes/Postal-codes-Norway-ansi.ts
src/minibootstrap.less
```

Out[11]:

module	code lines	code lines %
saksbehandling-ui	56419	37.565084
journalforing	19405	12.920301
prefill	18500	12.317731
fagmodul	13500	8.988614
pdl-produsent	7993	5.321926
ep-kodeverk	6020	4.008256
ep-eux	4507	3.000866
ep-personoppslag	3430	2.283774
statistikk	2961	1.971503
oppgave	2562	1.705839
saksbehandling-api	2400	1.597976
begrens-innsyn	2231	1.485452
meta	2143	1.426859
ep-routing	1829	1.217791
krav-initialisering	1694	1.127905
onprem-proxy	1145	0.762368
ep-logging	934	0.621879
ep-metrics	862	0.573940
ep-meta-analyse	850	0.565950
ep-pensjonsinformasjon	805	0.535988

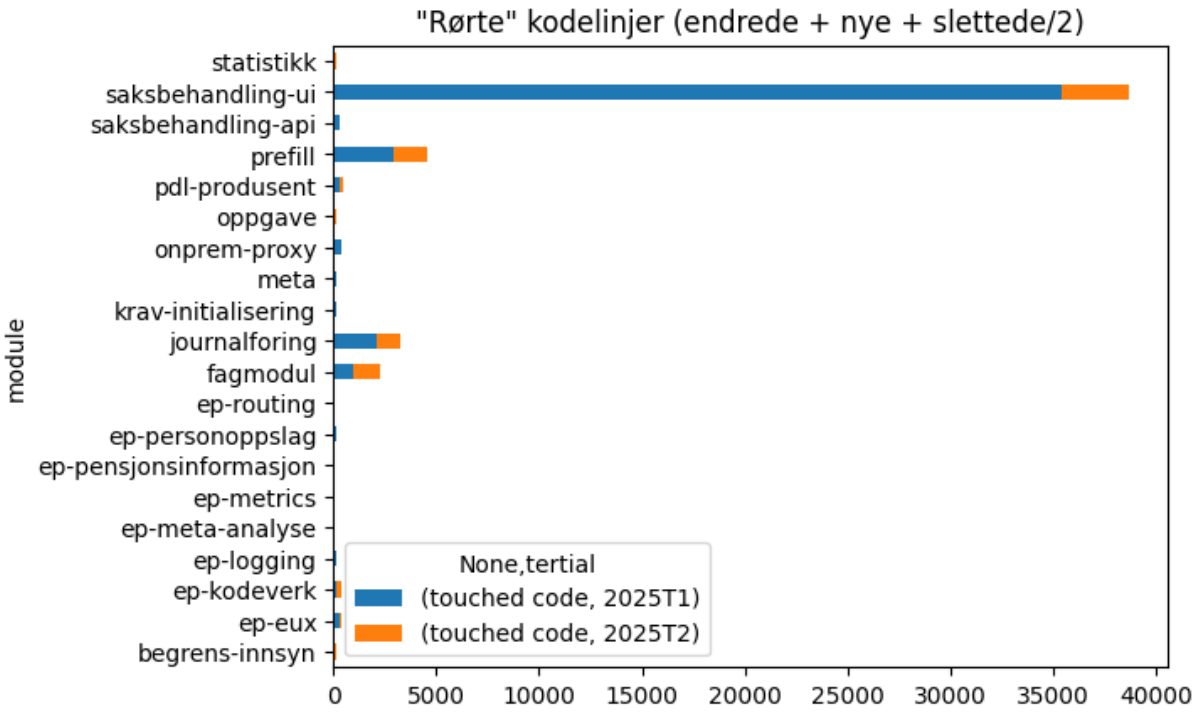
Frontend-app'en for saksbehandling utgjør om lag en tredel av den totale koden, i én app.





Kodeendringer

Vi kan se på hvilke app'er det er gjort kodeendringer i ved å sammenlikne koden på starten og slutten av tertialet.



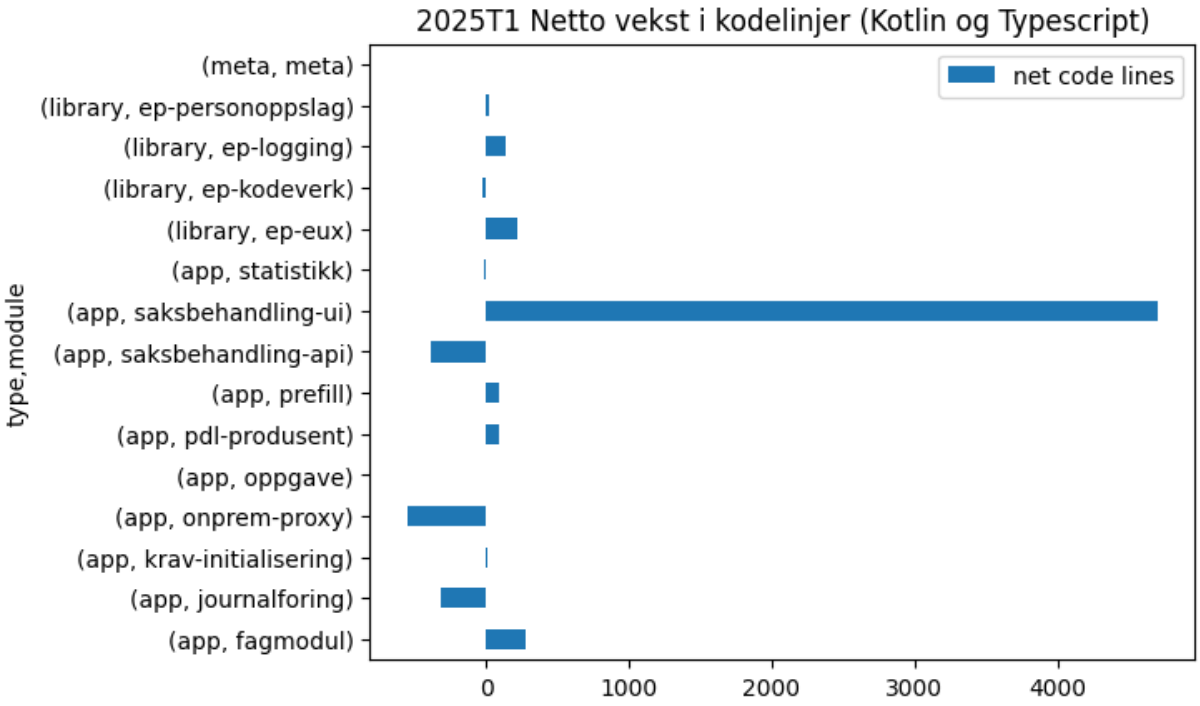
Vi ser ...

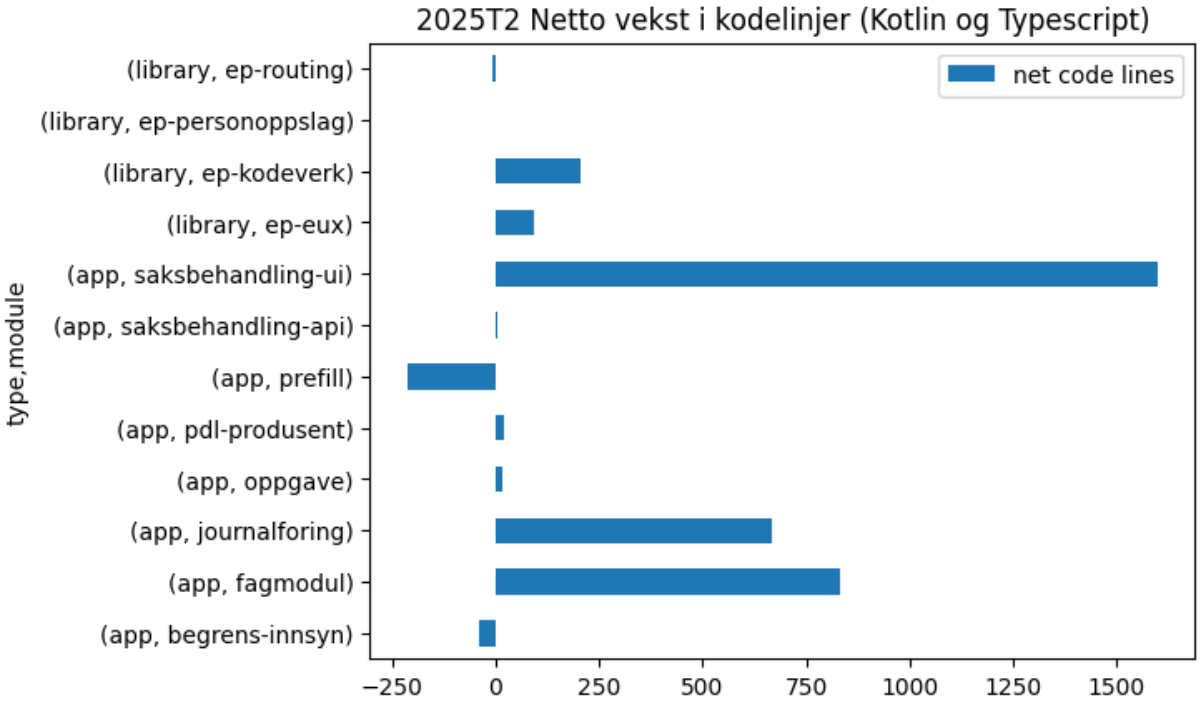
Hva betyr dette?

[TBW]

Netto vekst i kodelinjer

Netto vekst i kodelinjer indikerer at det blir mer kode å vedlikeholde. Refaktorering kan til en viss grad motvirke veksten.





Vi ser ...

Hva betyr dette?

[TBW]

Detaljer i netto vekst i kodelinjer

Out[16]:

language	Bourne Shell	Dockerfile	Gradle	GraphQL	JavaScript	Kotlin	Proper
module							
saksbehandling-ui	0	0	0	0	32	0	
fagmodul	0	0	5	0	0	832	
journalforing	0	0	5	0	0	667	
ep-kodeverk	0	0	-1	0	0	203	
ep-eux	0	0	-1	0	0	94	
oppgave	0	0	-2	0	0	15	
pdl-produsent	0	0	-2	0	0	21	
saksbehandling-api	0	0	-2	0	0	4	
krav-initialisering	0	0	-2	0	0	0	
statistikk	0	0	-2	0	0	0	
meta	3	0	-2	0	0	0	
ep-pensjonsinformasjon	0	0	-2	0	0	0	
ep-metrics	0	0	-2	0	0	0	
ep-logging	0	0	-2	0	0	0	
onprem-proxy	0	0	-5	0	0	0	
ep-personoppslag	0	0	-2	-7	0	1	
ep-routing	0	0	-2	0	0	-7	
begrens-innsyn	0	0	-2	0	0	-39	
prefill	0	0	-2	0	0	-213	
Total	3	0	-23	-7	32	1578	

Vi observerer at kodebasen netto har økt med ...

Dette er på tross av at ...

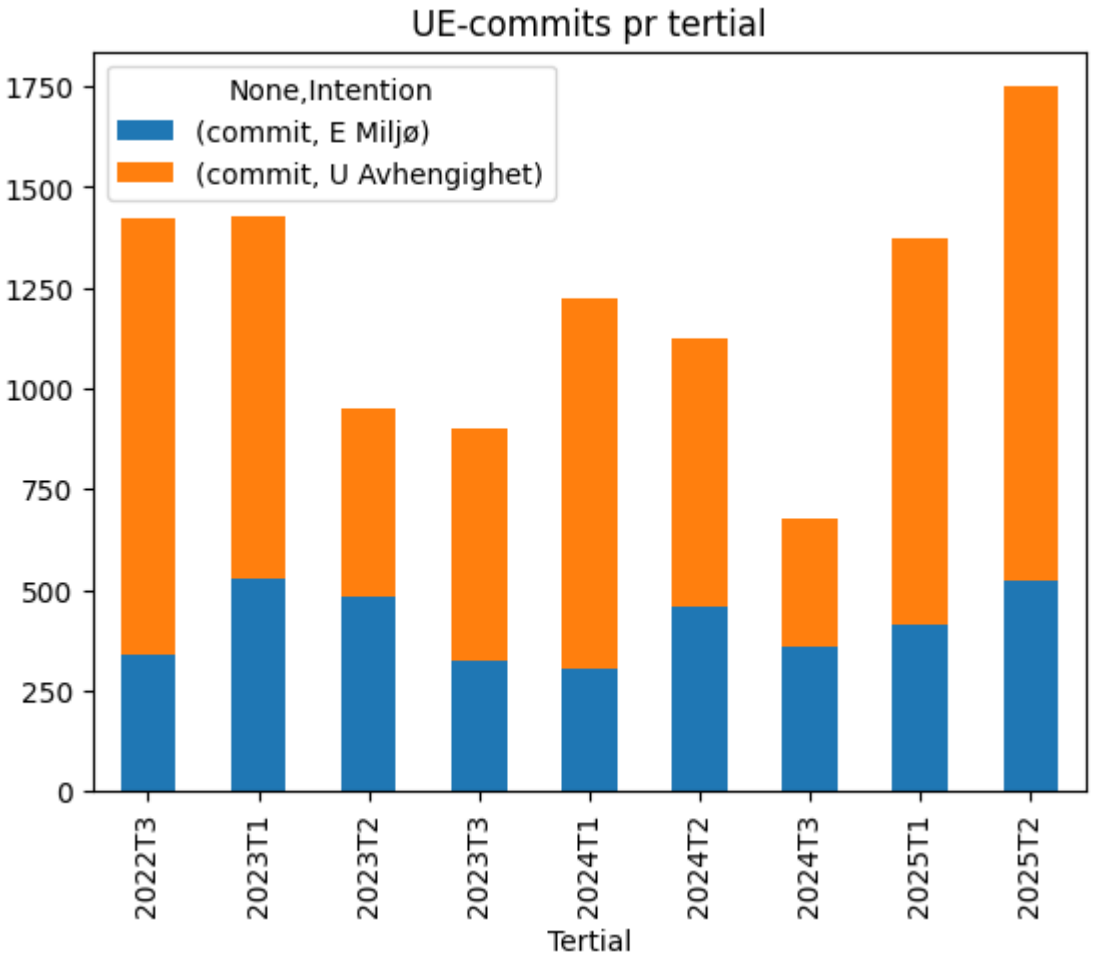
[TBW]

Hva betyr dette?

[TBW]

Oppdatering av avhengigheter

Oppdateringer av avhengigheter reduserer risiko for sikkerhetshull, og er med på å redusere problemene med sporadiske oppdateringer (mange ting som oppdateres samtidig, og at store hopp i versjoner gjør det vanskeligere å finne feil dersom noe går galt).



Vi observerer at [TBW]

Oppdateringer av avhengigheter pr modul og måned

Out[18]:

		commit				
	Måned	2025-05	2025-06	2025-07	2025-08	Total
type	module					
app	fagmodul	19	17	29	187	252
	oppgave	17	17	24	172	230
	pdl-produsent	21	22	32	15	90
	journalforing	18	18	27	13	76
	saksbehandling-api	17	19	21	12	69
	prefill	12	14	23	12	61
	statistikk	16	16	18	9	59
	krav-initialisering	16	16	17	9	58
	begrens-innsyn	13	14	20	8	55
	onprem-proxy	10	12	21	8	51
library	ep-personoppslag	11	14	13	8	46
	ep-routing	9	9	17	8	43
	ep-pensjonsinformasjon	7	10	9	5	31
	ep-kodeverk	3	9	9	5	26
app	saksbehandling-ui	1	3	9	9	22
library	ep-logging	4	7	5	3	19
	ep-eux	6	6	5	2	19
	ep-metrics	5	6	4	3	18

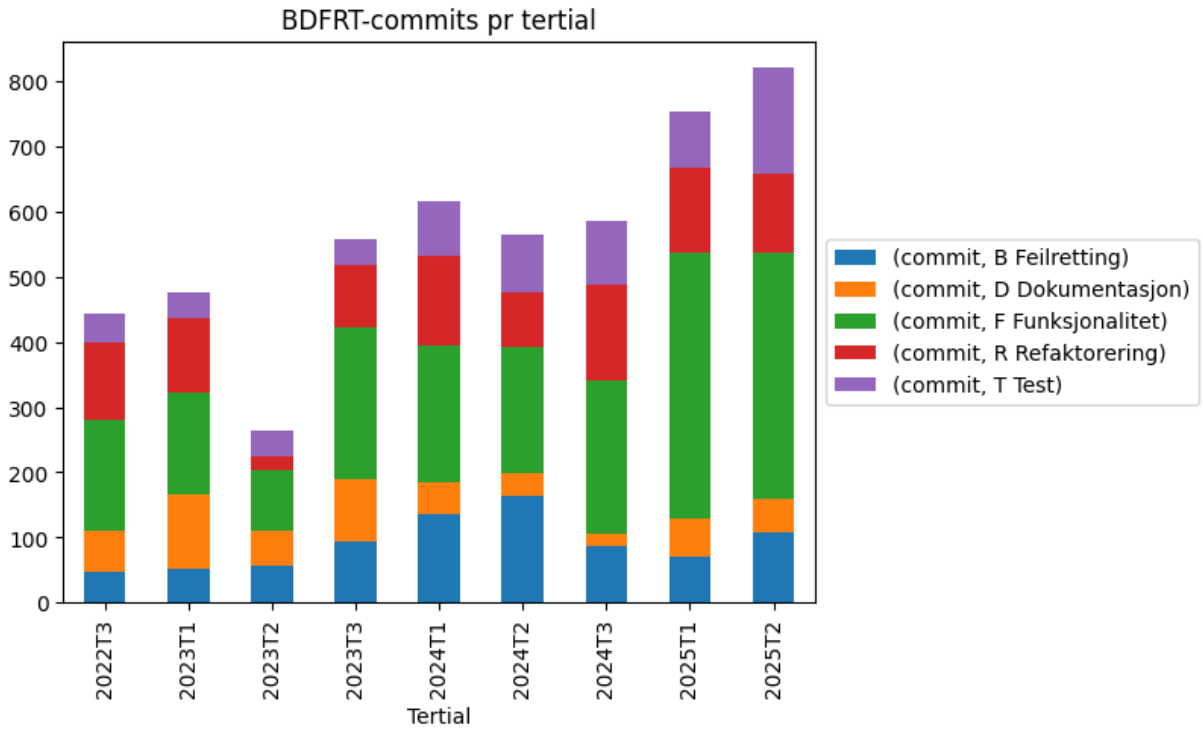
Det er ingen commits som kommer ut som oppdatering av avhengigheter for saksbehandling-ui - det skyldes trolig feil angivelse av intensjon (skulle vært angitt med intensjonen U), for det har skjedd oppdatering, om enn ikke så mange.

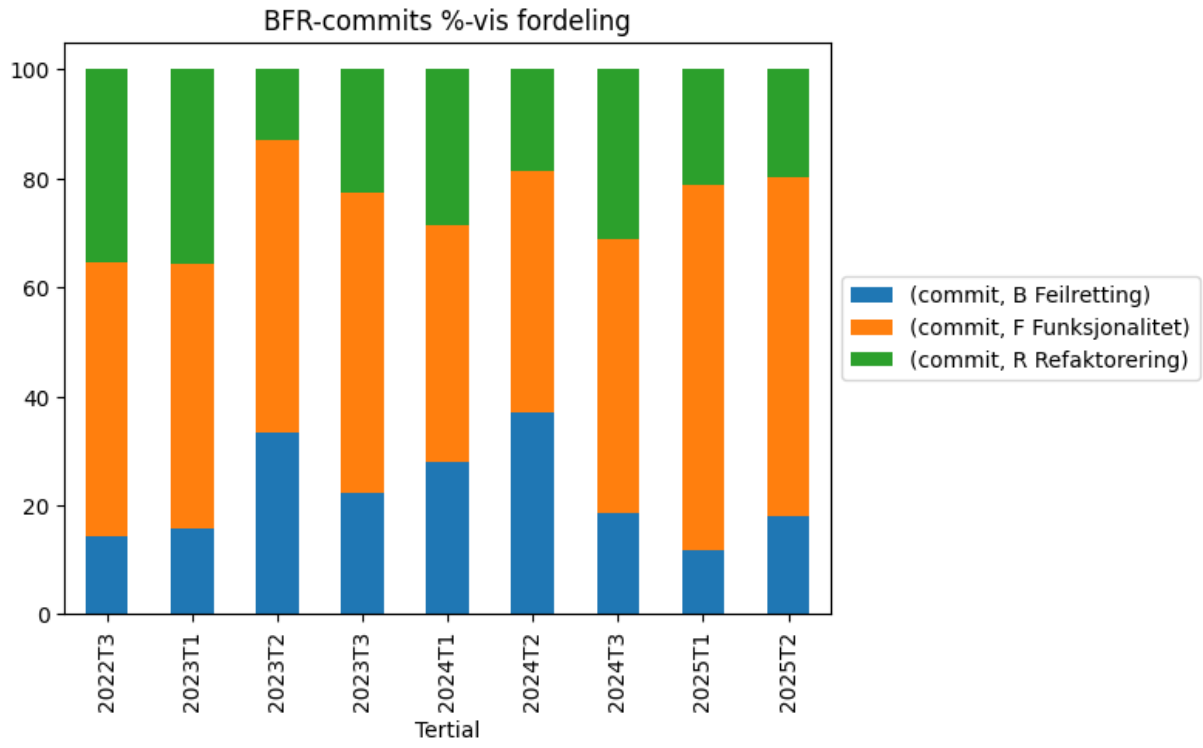
Hva betyr dette?

[TBW]

Løpende endringer i systemet

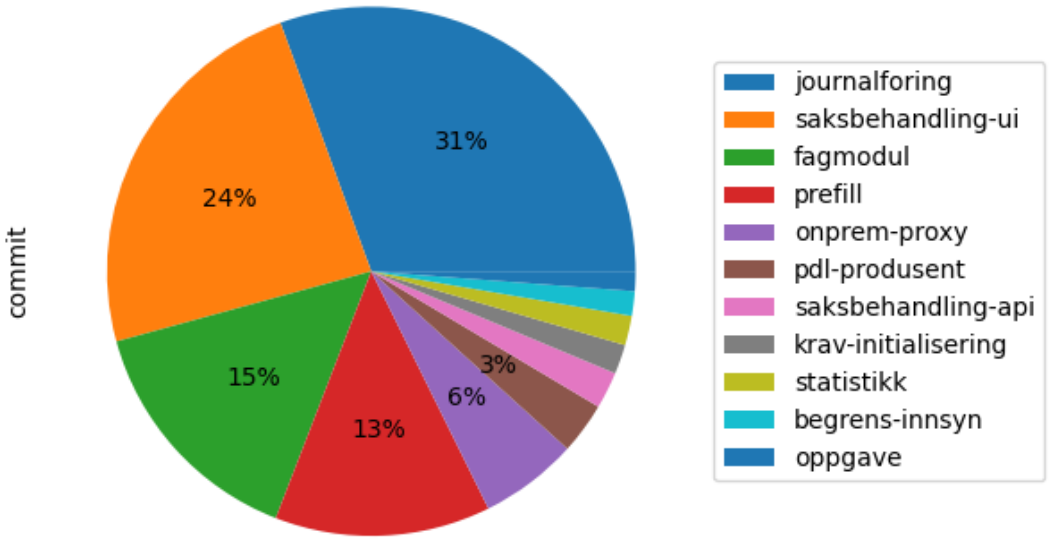
At det gjøres endringer i systemet er indikator på at systemet vedlikeholdes, noe som opprettholder kjennskapen til systemet. Vi ser her på endringer merket med B, D, F, R og T; de er knyttet til feilretting (B), dokumentasjon (nær/i koden) (D), funksjonalitet (F), interne forbedringer/refaktorering (R) og testforbedringer (T).



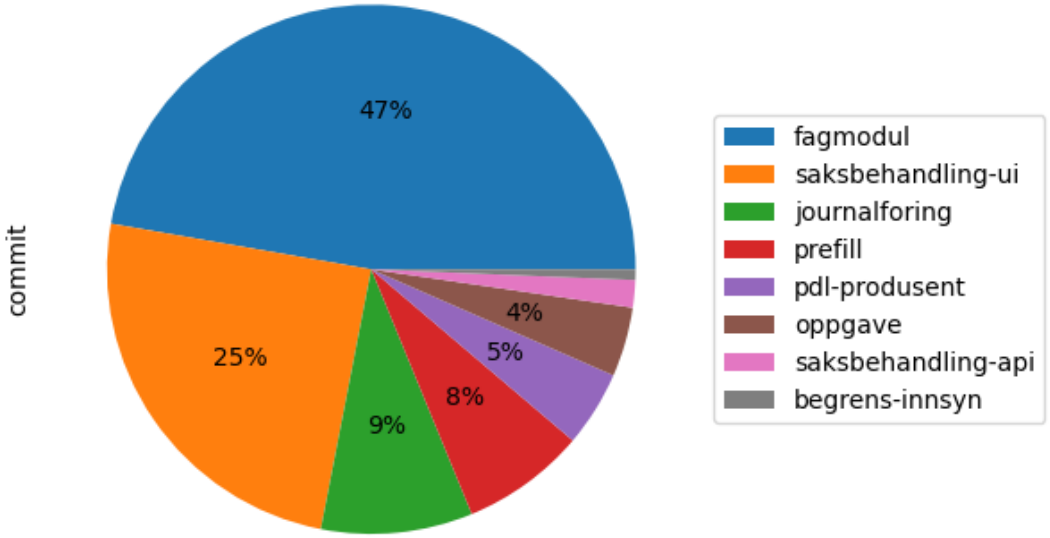


Vi observerer at [TBW]

Fordeling av BDFRT-commits på app-er (bibliotek er utelatt) i 2025T1



Fordeling av BDFRT-commits på app-er (bibliotek er utelatt) i 2025T2



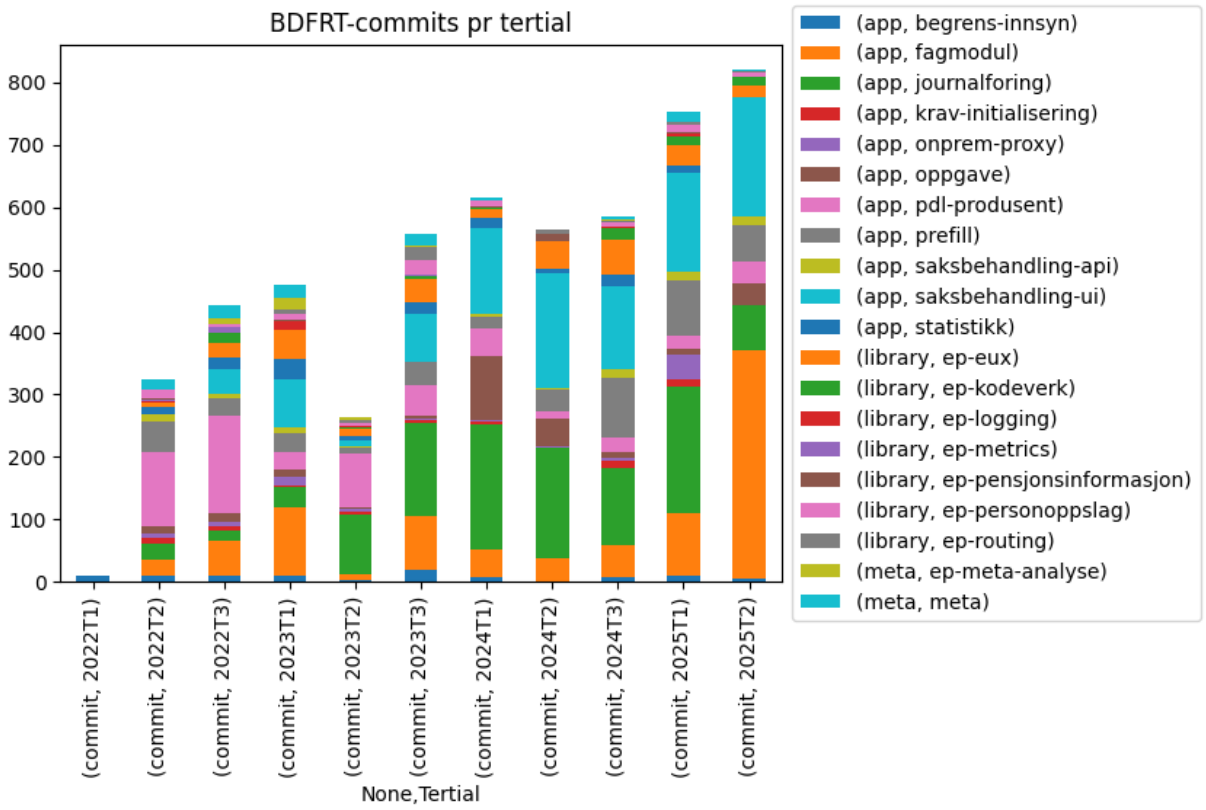
Vi observerer at [TBW]

BDFRT-oppdateringer pr modul

Out[22]:

		commit					Total
	intention	B	D	F	R	T	
type	module						
app	fagmodul	23	24	193	50	77	367
	saksbehandling-ui	45	0	79	25	43	192
	journalforing	12	10	21	18	11	72
	prefill	11	5	21	5	17	59
	pdl-produsent	15	3	17	1	0	36
	oppgave	0	1	18	7	7	33
	saksbehandling-api	1	1	6	3	2	13
	begrens-innsyn	0	0	1	2	2	5
library	ep-eux	2	1	8	3	3	17
	ep-kodeverk	0	3	8	5	0	16
	ep-personoppslag	0	1	3	1	0	5
	ep-routing	0	0	2	1	0	3
meta	ep-meta-analyse	0	1	0	0	0	1
	meta	0	0	1	0	0	1

Vi observerer at [TBW]



Vi observerer at [TBW]

Hva betyr dette?

[TBW]

Risiko

Vi merker commits med risiko, med en skala fra 0 til 5:

0. Ingen risiko
1. Kjent trygg
2. Validert
3. Kjent restrisiko
4. Feil/sannsynlig feil
5. Ukjent risiko

Endringer med risiko 1 Kjent trygg har svært lav risiko og krever lite eller ingen kjøring av tester. Dokumentasjons- og testendringer skal ha risiko 1. For endringer i produksjonskode er det strenge regler for hvilke endringer som kan merkes risiko 1 – stort sett går det ut på at det er små endringer som er gjort med støtte av funksjonalitet i utviklingsverktøyet (IDE).

Endringer med risiko 2 Validert krever som regel at tester kjøres lokalt for at man skal bli trygg, noe som kan ta litt tid, avhengig av app og oppsett. For oppdateringer av avhengigheter er det ofte vanskelig å si hva risiko er, men vi kjører tester for alle oppdateringer så vi merker dem rutinemessig med risiko 2.

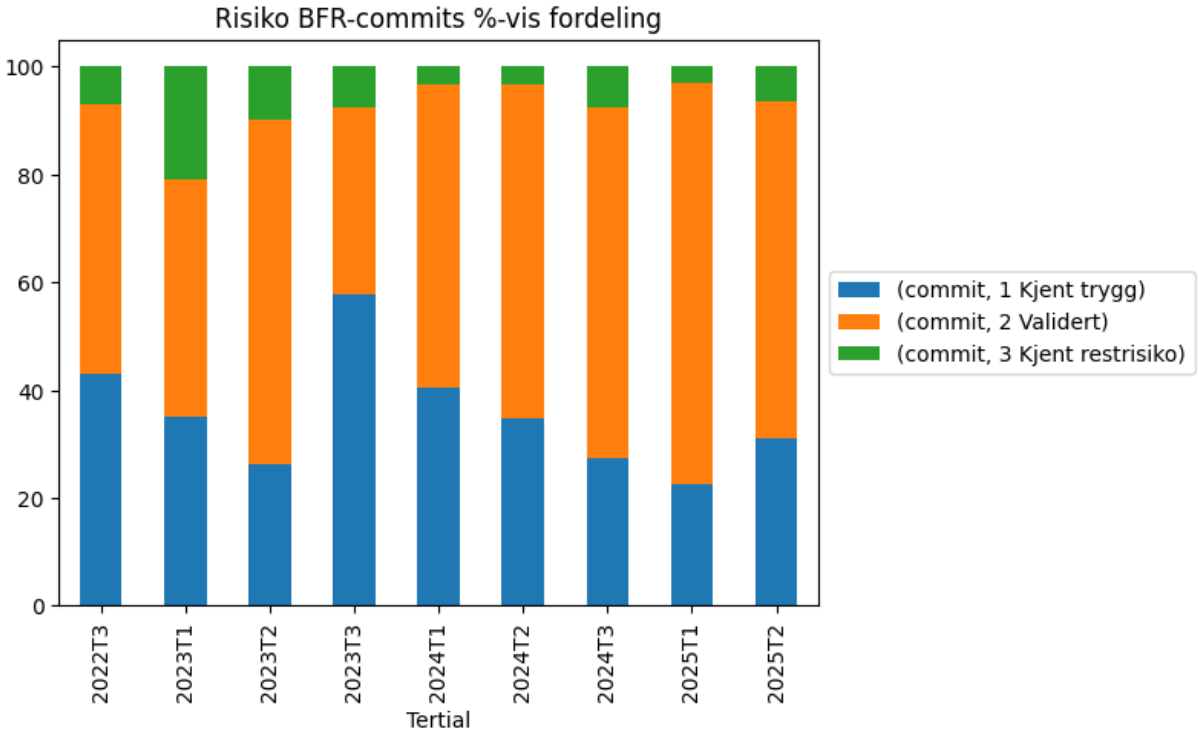
Endringer med risiko 3 Kjent restrisiko må ofte følges opp litt ekstra i forbindelse med produksjonssetting, eller de må testes manuelt før de går i produksjon.

Risiko 4 er normalt ikke akseptabelt å dytte til produksjon – dette er endringer som er uferdige eller har høy risiko.

Risiko 5 betyr at risiko er ukjent. Ofte vet man mer i situasjonen, men kunnskapen om risiko er tapt i ettertid (må eventuelt vurderes på nytt).

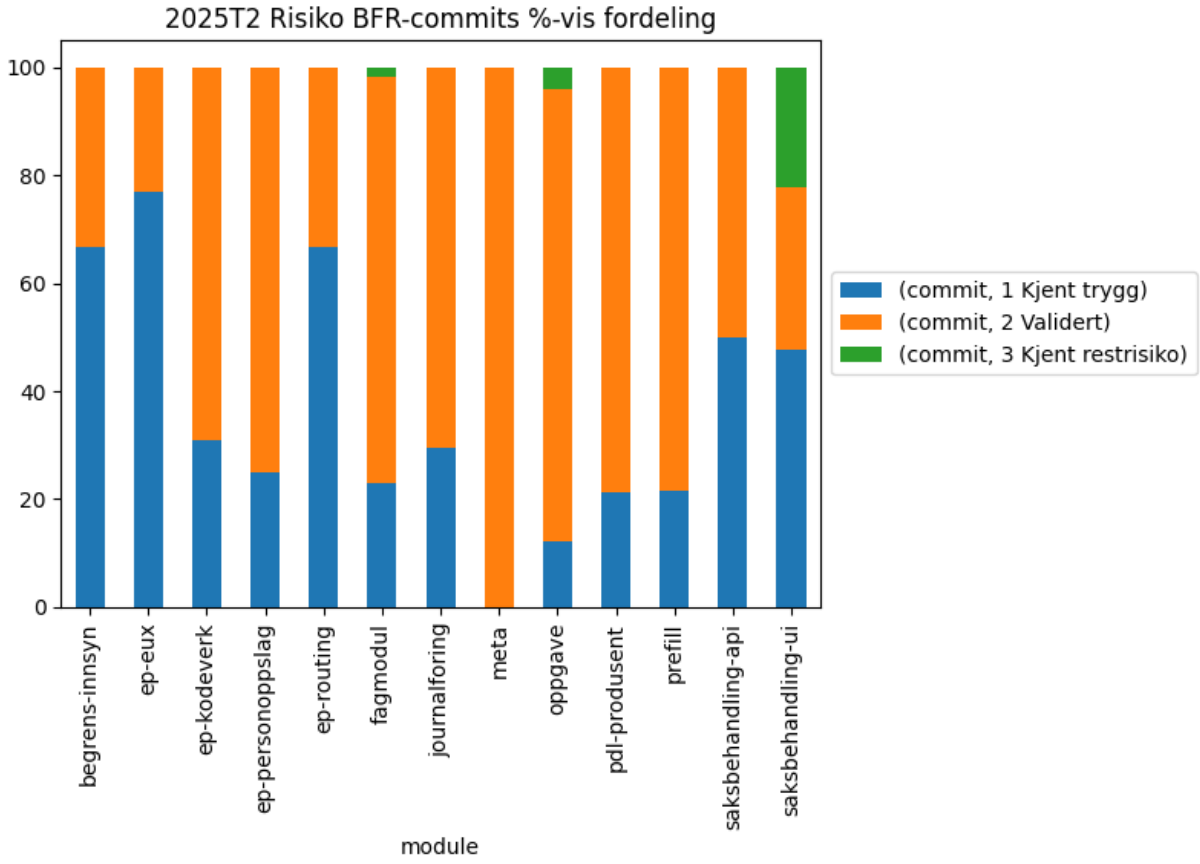
Det er nok litt mer slurv med vurdering av risikonivå enn det er med vurdering av intensjon.

De fleste endringer i produksjonskode vil være risiko fra 1 til 3



Vi observerer at [TBW]

Dette kan ha sammenheng med mer korrekt registrering av risiko, spesielt på frontend, der det ofte er restrisiko etter at en commit er gjort (som igjen innebærer at løsningen må testes manuelt, eller følges opp under produksjonssetting).



Hva betyr dette?

[TBW]